

Radiology Assistant (MediScan AI)

A Multi-Modal Deep Learning System for Brain Tumor, Pneumonia, COVID-19, and Kidney Disease Screening

Revised documentation — 4th diagnostic model (Kidney CT) added

Document Revision Summary

This documentation has been revised against the authoritative kidney_model_training.ipynb notebook to (1) incorporate the project's fourth diagnostic model — kidney CT classification (cyst / normal / stone / tumor) — throughout every relevant section, and (2) correct details that had drifted from the underlying implementation. The table below summarizes the sections most affected.

Section	Nature of change
2, 4	Project overview and objectives updated from three to four models; kidney explainability objective added.
5	Added CT Kidney dataset description, class counts, and split sizes.
6	Added kidney preprocessing row; clarified that no manual preprocess_input/rescale is applied because EfficientNet's built-in preprocessing layers handle this.
8	Added kidney augmentation configuration (wider rotation range, added vertical flip).
9–12	Added kidney model architecture, training schedule, hyperparameters, and loss/optimizer configuration, with exact parameter counts and callback settings taken from the notebook's execution log.
13–14	Added kidney evaluation metrics, full classification report, confusion matrix, error analysis, and a literature-benchmark comparison for context.
15–16	Added Grad-CAM + CLAHE explainability visualizations and the corresponding inference/interpretation pipeline — new to this model.
17–19	Added new libraries (split-folders, OpenCV), updated folder structure, and added kidney-specific implementation challenges, including a wording inconsistency between the notebook and the deployed stack.
20–21	Removed "add explainability" from Future Improvements (now implemented for kidney) and added follow-on items; updated the conclusion to reflect four models.

Contents

1. Project Title
 2. Project Overview
 3. Problem Statement
 4. Objectives
 5. Dataset Description
 6. Data Preprocessing
 7. Exploratory Data Analysis
 8. Data Augmentation
 9. Model Architecture(s)
 10. Training Process
 11. Hyperparameters
 12. Loss Function and Optimizer
 13. Evaluation Metrics
 14. Results
 15. Visualizations
 16. Inference / Prediction Process
 17. Libraries and Dependencies
 18. Project Folder Structure
 19. Challenges Encountered
 20. Future Improvements
 21. Conclusion
- Appendix: Data Sources Referenced in the Notebooks

1. Project Title

Radiology Assistant (MediScan AI) — Multi-Disease Medical Imaging Classification System combining four independently trained EfficientNet transfer-learning models (Brain MRI, Chest X-Ray, Chest CT, and Kidney CT) into a single Gradio web application.

Live deployment (three-model version confirmed): huggingface.co/spaces/PhilopateerSharl/radiology_assistant. As discussed in Sections 18 and 19, the kidney model is trained and exported by its notebook but its integration into this live Space as a fourth selectable scan type is not confirmed by the materials reviewed for this revision.

2. Project Overview

This project consists of four independent deep learning notebooks, each training a convolutional neural network (based on an EfficientNet backbone pretrained on ImageNet) to classify a different type of medical scan:

#	Notebook	Modality	Task	Backbone
1	EfficientNetB0__Brain_Tumor_MRI_Classification_3.ipynb	Brain MRI	4-class tumor classification	EfficientNetB0
2	covid_19.ipynb	Chest CT scan	Binary COVID-19 detection	EfficientNetB4*
3	Xray_Pneumonia_EfficientNetB3.ipynb	Chest X-Ray	Binary pneumonia detection	EfficientNetB3
4	kidney_model_training.ipynb	Kidney CT scan	4-class classification (cyst, normal, stone, tumor)	EfficientNetB0 (B3 configurable)

* The COVID-19 notebook's title and headers describe it as "EfficientNetB3," but the code instantiates EfficientNetB4 at 380×380 resolution; this documentation follows the architecture actually implemented in code, per the original documentation's own note.

The four trained model artifacts (best_brain_tumor_model.keras, x-ray.keras, Covid_19.keras, and — newly added — best_kidney_model.keras / kidney_efficientnetb0_final.keras) are intended to combine into a single production web application, MediScan AI, built with Gradio and deployed on Hugging Face Spaces. The app lets a user select a scan type, upload/photograph a medical image, and receive an AI-generated diagnosis, a confidence score, a severity rating, and plain-language medical guidance, alongside an integrated LLM-powered chatbot (Groq API, llama-3.3-70b-versatile).

New in this revision: the kidney notebook additionally implements a Grad-CAM-based visual explainability pipeline, combined with CLAHE contrast enhancement, that highlights the CT regions most responsible for each prediction (see Sections 15–16). This capability is currently specific to the kidney model; extending it to the other three models is listed as a future improvement (Section 20).

Minor inconsistency noted: the kidney notebook's own model-export cell refers to "your Streamlit app," whereas the project's actual deployment framework — confirmed elsewhere in this documentation and in the project description — is Gradio. This is treated as a wording slip in the notebook rather than a change of deployment stack.

3. Problem Statement

Manual interpretation of radiological images (MRI, X-ray, CT) requires trained specialists and is time-consuming, which can delay diagnosis — particularly in resource-constrained or high-volume clinical settings. Four specific, high-impact diagnostic tasks are addressed:

- Brain tumors (glioma, meningioma, pituitary tumor) are difficult to detect early, and misclassification (e.g., a true tumor being called "no tumor") carries serious clinical risk.
- Pneumonia requires timely diagnosis to begin correct antibiotic/antiviral treatment and reduce complications.
- COVID-19, particularly during periods of high caseload, benefits from CT-based screening as a complement to PCR/antigen testing.

- Kidney disease — cysts, stones, and tumors — is commonly identified on CT imaging; a well-imaged classifier able to distinguish between these findings and normal anatomy could help support and prioritize radiologist review, especially in the resource-constrained clinical settings the CT Kidney dataset itself was collected from (hospital PACS systems in Dhaka, Bangladesh, per the dataset's documentation).

The project explores whether transfer learning with EfficientNet architectures can produce automated, reasonably accurate screening tools for all four tasks, and whether such models can be unified into one accessible, patient-facing interface.

4. Objectives

Per-notebook objectives (as stated/implemented in the notebooks):

- Load and preprocess medical images (MRI / CT / X-ray) for each condition.
- Apply appropriate data augmentation to improve generalization.
- Build EfficientNet-based transfer-learning classifiers.
- Train models in two phases: head-only training followed by fine-tuning.
- Evaluate performance using accuracy, precision, recall, F1-score, confusion matrices, and ROC-AUC.
- Visualize training curves (accuracy/loss) and classification results.
- Save the best-performing model checkpoint for deployment.
- (Kidney model, new) Produce Grad-CAM visual explanations, enhanced with CLAHE contrast adjustment, for one representative image per class.
- (Kidney model, new) Contextualize the achieved test accuracy against independently published benchmarks on the same public dataset.

Project-level (deployment) objectives:

- Unify all four trained models into a single diagnostic web application.
- Provide patients/users with an understandable diagnosis, confidence level, and recommended next steps.
- Offer an AI chatbot for follow-up questions about the diagnosed condition.
- Maintain a session-based scan history log.
- Clearly disclose that the tool is for educational/research purposes and is not a medical device.
- (New) Provide region-level visual explanations for model predictions, beginning with the kidney model, to support user and clinician trust in the tool's outputs.

5. Dataset Description

Dataset	Source	Classes	Total Images	Train / Val / Test
Brain Tumor MRI Dataset	Kaggle (brain-tumor-mri-dataset)	glioma, meningioma, notumor, pituitary	7,200	5,600 / — / 1,600
SARS-CoV-2 CT-Scan Dataset	Kaggle, plameneduardo/sarscov2-ctscan-dataset	COVID, non-COVID	2,481	1,744 / 496 / 241
Chest X-Ray Images (Pneumonia)	Kaggle, paultimothymooney/chest-xray-pneumonia	NORMAL, PNEUMONIA	5,840+624	4,173 / 1,043 / 624
CT KIDNEY DATASET: Normal-Cyst-Tumor-and-Stone	Kaggle, nazmul0087/ct-kidney-dataset-normal-cyst-tumor-and-stone	Cyst, Normal, Stone, Tumor	12,446	8,710 / 1,865 / 1,871

Kidney dataset class counts: Cyst 3,709 · Normal 5,077 · Stone 1,377 · Tumor 2,283. Per the dataset's own published documentation, these images were sourced from hospital PACS systems in Dhaka, Bangladesh, and verified by a

radiologist; this provenance claim originates from the dataset source and was not independently re-verified within this project.

Notes:

- The Brain Tumor dataset is perfectly class-balanced, so no class weighting was required.
- The COVID-19 dataset was split 70%/20%/10% at the batch level.
- The Pneumonia dataset uses an 80/20 split of the train folder plus an independent 624-image test folder.
- The Kidney dataset shows the most pronounced class imbalance of the four (Stone at 1,377 images is roughly 3.7× rarer than Normal at 5,077) and was split 70/15/15 using the split-folders utility, which partitions each class independently (seed=42) to preserve class proportions across all three sets — a different splitting mechanism from the other three notebooks, which relied on framework-native directory splitting.
- All three imbalanced datasets (COVID, Pneumonia, Kidney) were corrected using `sklearn.utils.class_weight.compute_class_weight(class_weight="balanced", ...)`. For Kidney specifically, the computed weights are: Cyst 0.839, Normal 0.613, Stone 2.261, Tumor 1.363.

6. Data Preprocessing

Step	Brain Tumor (B0)	COVID CT (B4)	Pneumonia X-Ray (B3)	Kidney CT (B0/B3)
Image loading	<code>ImageDataGenerator.flow_from_directory</code>	<code>image_dataset_from_directory</code>	<code>image_dataset_from_directory</code>	<code>ImageDataGenerator.flow_from_directory</code>
Input resolution	224×224×3	380×380×3	384×384×3	224×224×3 (B0) / 300×300×3 (B3)
Batch size	32	16	16	32
Pixel scaling	<code>efficientnet.preprocess_input</code> via <code>preprocessing_function</code>	<code>preprocess_input</code> as a graph layer	<code>preprocess_input</code> as a graph layer, post-augmentation	None applied manually — raw [0,255]; EfficientNet's built-in Rescaling+Normalization layers handle this internally
Label encoding	Categorical (one-hot, 4 classes)	Binary (<code>label_mode="binary"</code>)	Binary (<code>label_mode="binary"</code>)	Categorical (one-hot, 4 classes)
Class imbalance handling	Not needed (balanced)	<code>class_weight="balanced"</code>	<code>class_weight="balanced"</code>	<code>class_weight="balanced"</code>
Pipeline optimization	N/A (generator-based)	<code>.cache().prefetch(AUTOTUNE)</code>	<code>.prefetch(AUTOTUNE)</code>	N/A (generator-based)
Reproducibility	seed=42 (TF, NumPy, generators)	seed not explicitly fixed at split step	seed=42 in dataset splits	seed=42 (Python, NumPy, TF) plus seed=42 in split-folders and the training generator

Clarification: the kidney notebook's own commentary attributes the lack of a manual `preprocess_input` call to targeting "Keras 3." In fact, `keras.applications.EfficientNet*` backbones have included built-in Rescaling and Normalization layers since their original release, independent of the Keras 3 upgrade — what matters operationally is simply that no additional `preprocess_input` or `rescale=1./255` step should be layered on top, since doing so would double-normalize the inputs and degrade accuracy (the same class of bug previously observed on the Brain Tumor model, per Section 19 of the original documentation). The kidney notebook's actual code correctly avoids this, regardless of the stated rationale.

7. Exploratory Data Analysis

EDA in these notebooks is primarily visual rather than statistical:

- Brain Tumor notebook: prints the unzipped folder tree and per-class image counts, and confirms the generator output (Found 5,600 / 1,600 images belonging to 4 classes).
- COVID-19 notebook: prints Found 2,481 files belonging to 2 classes and computes/prints the class-weight dictionary.
- Pneumonia notebook: walks the downloaded Kaggle directory to print its folder structure and prints the train/validation/test split counts.
- Kidney notebook: locates the (potentially nested) dataset root directory programmatically, prints raw per-class file counts (`{'Cyst': 3709, 'Normal': 5077, 'Stone': 1377, 'Tumor': 2283}`), visualizes this distribution as a labeled bar chart, prints the split-folders output (train: 8,710 / val: 1,865 / test: 1,871 images), confirms the generator output (Found 8,710 / 1,865 / 1,871 images belonging to 4 classes), and prints the resolved class-to-index mapping (`{'Cyst': 0, 'Normal': 1, 'Stone': 2, 'Tumor': 3}`).

As with the other three notebooks, no statistical/tabular EDA (pixel-intensity distributions, image-size histograms) is present in the kidney notebook; class balance is assessed purely from file counts.

8. Data Augmentation

Notebook	Augmentation applied	Method
Brain Tumor	Horizontal flip, rotation $\pm 15^\circ$, width/height shift $\pm 10\%$, shear 10%, zoom 10%, brightness $\pm 20\%$	ImageDataGenerator, training data only
COVID-19 CT	None	Class imbalance instead handled via computed class weights
Pneumonia X-Ray	Random rotation ± 0.05 , random zoom 10%, random contrast 10%	Sequential augmentation layer, first layer inside the model graph
Kidney CT	Rotation $\pm 30^\circ$, width/height shift $\pm 10\%$, shear 10%, zoom 15%, brightness range 0.85–1.15, horizontal AND vertical flip	ImageDataGenerator, training data only; validation/test generators apply no augmentation and no manual rescale

The kidney notebook's augmentation ranges are the widest of the four models — notably the $\pm 30^\circ$ rotation range (versus $\pm 15^\circ$ for Brain Tumor) and the addition of vertical flipping, which none of the other three models use. Per the notebook's own documentation, these specific ranges were chosen to mirror values reported in a prior published two-stage EfficientNetB2 study on this same dataset. A single 8-image grid of augmented training samples spanning all four classes is generated for a visual sanity check; unlike the Brain Tumor notebook, no separate pre-augmentation sample grid is saved to disk.

9. Model Architecture(s)

All four models follow the same general transfer-learning pattern: a frozen/partially-frozen EfficientNet backbone (ImageNet weights) → GlobalAveragePooling2D → BatchNormalization → Dense (ReLU) → Dropout → classification head.

9.1 Brain Tumor Model — EfficientNetB0

Input (224, 224, 3) → EfficientNetB0 (238 layers, no top) → GlobalAveragePooling2D (1280,) → BatchNormalization → Dense(256, ReLU) → Dropout(0.5) → Dense(4, softmax) → [glioma, meningioma, notumor, pituitary]. Total parameters: 4,383,675; trainable (Phase 1): 331,524.

9.2 COVID-19 CT Model — EfficientNetB4

Input (380, 380, 3) → preprocess_input → EfficientNetB4 (no top) → GlobalAveragePooling2D → BatchNormalization → Dropout(0.4) → Dense(256, ReLU) → Dropout(0.3) → Dense(1, sigmoid). Total parameters: 18,140,256; trainable (Phase 1): 462,849; non-trainable: 17,677,407.

9.3 Pneumonia X-Ray Model — EfficientNetB3

Input (384, 384, 3) → Data augmentation → preprocess_input → EfficientNetB3 (no top) → GlobalAveragePooling2D → BatchNormalization → Dropout(0.4) → Dense(256, ReLU) → Dropout(0.3) → Dense(1, sigmoid). Total parameters: 11,183,408; trainable (Phase 1): 396,801; non-trainable: 10,786,607.

9.4 Kidney Disease Model — EfficientNetB0 (new)

Input (224, 224, 3) [300×300×3 if configured for EfficientNetB3] → EfficientNetB0 (ImageNet weights, no top) → GlobalAveragePooling2D → (1280,) → BatchNormalization → Dropout(0.35) → Dense(256, activation="relu", kernel_regularizer=L2(1e-4)) → Dropout(0.25) → Dense(4, activation="softmax", name="predictions") → [Cyst, Normal, Stone, Tumor].

Total parameters: 4,383,655. Trainable parameters (Phase 1, backbone frozen): 331,524. Non-trainable parameters (Phase 1): 4,052,131.

Design notes: the kidney model follows the same GAP → BatchNorm → Dropout → Dense head pattern as the Brain Tumor model, but introduces two differences not present in any of the other three models: (1) L2 kernel regularization (1e-4) is applied to the Dense(256) layer, and (2) two distinct dropout rates are used (0.35 before the dense layer, 0.25 after) rather than a single Dropout(0.5). The base model is also called with training=False explicitly, which keeps its BatchNormalization sub-layers in inference mode both before and after the backbone is partially unfrozen in Phase 2 — the notebook documents this as the recommended way to fine-tune EfficientNet without destabilizing its batch-statistics.

The build_model() function is parameterized to build either an EfficientNetB0 or EfficientNetB3 backbone from a single MODEL_NAME configuration flag, allowing the two backbones to be compared by re-running the notebook from the model-build cell onward. The results reported throughout this documentation correspond to the EfficientNetB0 configuration, which is the one actually trained and evaluated in the notebook as executed.

10. Training Process

All four models use a two-phase transfer-learning strategy: (1) train only the custom classification head while the backbone is frozen, then (2) unfreeze part of the backbone and fine-tune end-to-end at a much lower learning rate.

Phase	Brain Tumor (B0)	COVID CT (B4)	Pneumonia (B3)	Kidney (B0)
Phase 1 (head only)	15 epochs, frozen, LR=1e-3	7 epochs, frozen, LR=1e-4	25 epochs, frozen, LR=1e-4	15 epochs (budget, fully used, no early stop), frozen LR=1e-3
Phase 2 (fine-tune)	35 epochs, top 138/238 layers unfrozen, LR=1e-5	3 epochs, last 80 unfrozen, LR=1e-5	10 epochs, last 50 unfrozen, LR=1e-5	Up to 40 epochs (budget); last 72/238 layers unfrozen (30%); LR=1e-4; early-stopped at epoch 12 (patience 6 on val_loss); best weights restored from epoch 6
Extra refinement	Additional 20-epoch "Enhanced Phase 2" pass	—	—	None — single fine-tuning pass
Class weighting	Not needed (balanced)	class_weight="balanced"	class_weight="balanced"	class_weight="balanced" (both phases)

All models were trained on a GPU-accelerated runtime; the kidney notebook confirms TensorFlow 2.20.0, Keras 3.13.2, and an available NVIDIA T4 GPU. In Phase 2, ReduceLROnPlateau decayed the kidney model's learning rate from 1e-4 to 5e-5 at epoch 9 and to 2.5e-5 at epoch 12, immediately before early stopping triggered. Notably, the kidney model's ModelCheckpoint-saved best epoch (Phase-2 epoch 6, val_accuracy = 0.9861) coincides exactly with the epoch EarlyStopping restores (also epoch 6, val_loss = 0.1063), so the two mechanisms agree on the same set of weights for this model — unlike some ambiguity that can arise when the two callbacks monitor different metrics.

11. Hyperparameters

Hyperparameter	Brain Tumor	COVID-19 CT	Pneumonia X-Ray	Kidney CT
Image size	224×224	380×380	384×384	224×224 (B0) / 300×300 (B3)
Batch size	32	16	16	32
Phase-1 epochs	15	7	25	15 (budget, fully used)
Phase-2 epochs	35 (+20 enhanced)	3	10	40 (budget) / 12 run (early-stopped)
Phase-1 LR	1e-3	1e-4	1e-4	1e-3
Phase-2 LR	1e-5	1e-5	1e-5	1e-4 (decayed to 5e-5, then 2.5e-5)
Dropout	0.5 (single)	0.4, 0.3	0.4, 0.3	0.35, 0.25
Regularization	—	—	—	L2 (1e-4) on Dense(256)
Label smoothing	0.10 → 0.05 (enhanced)	—	—	None used
Random seed	42	42 (partial)	42	42 (global + split-folders + generator)

Shared training callbacks for the kidney model: ModelCheckpoint (monitor val_accuracy, save_best_only=True), EarlyStopping (monitor val_loss, patience=6, restore_best_weights=True), and ReduceLROnPlateau (monitor val_loss, factor=0.5, patience=3, min_lr=1e-7) — used identically across both training phases. This is a more tightly and consistently specified callback configuration than the range described for the other three models ("patience 2–4 epochs," "factor 0.5 or 0.2").

12. Loss Function and Optimizer

Model	Optimizer	Loss Function
Brain Tumor	Adam	CategoricalCrossentropy(label_smoothing=0.1), reduced to 0.05 in enhanced fine-tuning
COVID-19 CT	Adam	binary_crossentropy
Pneumonia X-Ray	Adam	binary_crossentropy
Kidney CT	Adam	categorical_crossentropy (no label smoothing)

All four models track accuracy as a training metric. COVID and Pneumonia additionally track AUC. The kidney model tracks the widest set of metrics during training — accuracy, AUC, Precision, and Recall simultaneously — the only one of the four models to monitor Precision and Recall directly at training time rather than computing them only at post-hoc evaluation.

13. Evaluation Metrics

- Accuracy — overall proportion of correct predictions.
- Precision, Recall, F1-score — per class, via sklearn.metrics.classification_report.
- Confusion Matrix — raw counts; the Kidney and Brain Tumor models additionally produce a row-normalized percentage version.
- ROC-AUC — one-vs-rest per class for the 4-class Brain Tumor and Kidney models; single binary ROC-AUC for COVID and Pneumonia.

- Custom composite "Ranking Score" (COVID notebook only): $0.4 \times \text{Accuracy} + 0.3 \times \text{F1} + 0.3 \times \text{ROC-AUC}$. The Kidney notebook reports its Accuracy, Precision, Recall, F1, and AUC individually rather than combining them into one composite score.
- (Kidney model, new) A qualitative Grad-CAM evaluation, providing region-level visual evidence of what the model attended to for one representative image per class, in addition to the quantitative metrics above.

Recall was explicitly called out in the Brain Tumor notebook as the most clinically important metric. The kidney notebook does not make an equivalent explicit statement, but its class weighting and per-class recall reporting serve the same underlying purpose of surfacing under-detected classes.

14. Results

14.1 Brain Tumor MRI Classification (EfficientNetB0)

Test set: 1,600 images (400 per class). Overall accuracy 95.31% (95.63% after enhanced fine-tuning); macro AUC 0.9908; macro F1 0.9521. Weakest metric: glioma recall (83.5%).

14.2 COVID-19 CT Classification (EfficientNetB4)

Test set: 241 CT images. Overall accuracy 91.29%; ROC-AUC 97.14%; composite ranking score 0.9306. COVID recall (87%) is lower than its precision (96%), meaning some true COVID cases are missed.

14.3 Pneumonia X-Ray Classification (EfficientNetB3)

Test set: 624 X-ray images. Overall accuracy 89.42% (execution-log figure) / 91.51% (notebook's own narrative summary, likely reflecting the best-checkpoint vs. final-run discrepancy noted in Section 19); ROC-AUC 95.15%.

14.4 Kidney Disease Classification (EfficientNetB0) — new

Test set: 1,871 CT images (Cyst 557, Normal 763, Stone 208, Tumor 343).

Class	Precision	Recall	F1-score	Support
Cyst	0.9721	1.0000	0.9858	557
Normal	0.9960	0.9685	0.9821	763
Stone	0.9050	0.9615	0.9324	208
Tumor	1.0000	0.9767	0.9882	343

Overall accuracy: 97.86% (1,871 images). Macro average: precision 0.9683, recall 0.9767, F1 0.9721. Weighted average: precision 0.9795, recall 0.9786, F1 0.9788. Macro-average one-vs-rest ROC-AUC: 0.9994. Test loss: 0.1218.

Confusion matrix (rows = actual, columns = predicted; class order Cyst, Normal, Stone, Tumor):

Actual \ Predicted	Cyst	Normal	Stone	Tumor
Cyst	557	0	0	0
Normal	4	739	20	0
Stone	8	0	200	0
Tumor	4	3	1	335

Error analysis: Cyst achieves perfect recall (no cyst case is missed), though its precision (0.9721) is slightly reduced by 8 Stone and 4 Normal cases being misclassified as Cyst. The single weakest metric overall is Stone precision (0.9050): of the 221 scans the model predicted as Stone, 20 were actually Normal and 1 was actually a Tumor — making Stone the class the model most often over-predicts. Tumor achieves perfect precision (no other class is ever misclassified as Tumor), though 8 of 343 true tumors are missed (predicted instead as Cyst, Normal, or Stone).

14.5 Comparison with Published Benchmarks on the Same Dataset

Several independently published studies report results on this exact CT Kidney dataset, providing external context for the 97.86% accuracy achieved here. A modified EfficientNetB3-based model ("Kid-Net") reported 99.30% cross-validation accuracy; a fine-tuned EfficientNetV2B0 model reported 99.73–99.75% accuracy; KidneyNeXt, a custom lightweight multi-branch CNN, reported 99.96% accuracy; and, most directly comparable, plain EfficientNet-B1 and EfficientNet-B2 baselines (without further architectural modification) reported 98.07% and 98.61% accuracy respectively on this dataset. Full citations are listed in the Appendix.

These figures indicate some headroom above the 97.86% achieved by this project's straightforward two-phase transfer-learning approach. The higher-scoring published results generally involve either custom architectural modifications (KidneyNeXt), dedicated class-rebalancing via data trimming (EfficientNetV2B0 study), or cross-validation rather than a single held-out test split (Kid-Net), rather than a plain EfficientNet backbone trained the way this project's model was. These figures are included only as external reference points for interpretation; they were not reproduced or independently re-verified within the scope of this project.

14.6 Cross-Model Summary

Model	Task	Accuracy	ROC-AUC	F1 (macro)
Brain Tumor (EfficientNetB0)	4-class MRI classification	95.31% (95.63% enh.)	0.9908	0.9521
COVID-19 (EfficientNetB4)	Binary CT classification	91.29%	0.9714	0.91
Pneumonia (EfficientNetB3)	Binary X-ray classification	89.42%	0.9515	0.89
Kidney Disease (EfficientNetB0)	4-class CT classification	97.86%	0.9994	0.9721

15. Visualizations

Notebook	Plots generated
Brain Tumor	Augmented sample grid; combined accuracy/loss curves across both phases; dual confusion matrix (raw + row-normalized); grouped per-class precision/recall/F1 bar chart; one-vs-rest ROC curves; gallery of misclassified images.
COVID-19	Raw sample grid; train/val accuracy curve; train/val loss curve; confusion matrix heatmap.
Pneumonia	Raw sample grid; train/val accuracy curve; train/val loss curve; confusion matrix heatmap; side-by-side inference panel for 2 test images.
Kidney CT (new)	Class-distribution bar chart; 8-image grid of augmented training samples across all four classes; combined Phase-1+Phase-2 accuracy/loss curves; raw-count confusion matrix heatmap (Blues); row-normalized "Diagnostic Reliability" confusion matrix heatmap (Greens, percentage-annotated); one-vs-rest ROC curves with per-class AUC in the legend; and Grad-CAM explainability figures — a 3-panel view per class showing the raw CT scan, a CLAHE-enhanced version, and the Grad-CAM heatmap overlay, titled with the predicted class and confidence (green if correct, red if incorrect).

As with the COVID-19 and Pneumonia notebooks, none of the kidney notebook's figures are persisted to disk — all are displayed inline via `plt.show()` only, unlike the Brain Tumor notebook's `./results/` folder.

16. Inference / Prediction Process

Within the notebooks: the Brain Tumor notebook defines a full single-image inference pipeline including an experimental "is this a brain scan?" safety layer and a plain-language explanation helper. The kidney notebook adds two comparable pieces of new functionality:

- `predict_and_explain_single()`: loads and resizes a single image to the configured input size, runs a forward pass, and prints a plain-language diagnosis block — predicted condition, confidence, and a recommended next step ("Consult Radiologist for {condition} verification").
- `get_gradcam_heatmap()` / `visualize_all_classes_gradcam()`: a full Grad-CAM explainability pipeline. Because Keras 3's functional-model API does not allow directly retrieving an arbitrary intermediate layer's output from a nested sub-model in the same way older Keras versions did, the implementation explicitly reconstructs a sub-model whose input is the EfficientNet backbone's own input tensor and whose outputs are (a) the target convolutional layer (`top_activation`) and (b) the backbone's final feature map. It then manually re-applies the five classification-head layers (`GlobalAveragePooling2D` → `BatchNormalization` → `Dropout` → `Dense(256)` → `Dropout` → `predictions`) inside a `tf.GradientTape` context to obtain the gradient of the predicted class's output with respect to the target convolutional feature map. These gradients are global-average-pooled per channel, used to weight the feature map, passed through `ReLU`, and normalized to `[0, 1]` — the standard Grad-CAM formulation, adapted here for a Keras 3 nested functional model.

For visualization only, the raw CT image is separately enhanced with CLAHE (Contrast Limited Adaptive Histogram Equalization), applied to the L-channel in LAB color space via OpenCV, purely to improve human interpretability of the surrounding anatomy. CLAHE enhancement is not used during training or as part of the model's actual input pipeline — only in the explainability visualization step, where the Grad-CAM heatmap (rendered with the Jet colormap) is alpha-blended (60% CLAHE image / 40% heatmap) over the enhanced image.

In the deployed application (per the original three-model documentation): all trained .keras models are loaded once at startup; the user selects a scan type, enters patient details, and uploads/photographs an image; the image is resized to the correct input size and passed through the matching model; a probability safety-net function normalizes raw outputs that may not already be valid probability distributions; and the result is rendered as a confidence-aware diagnosis card. The kidney model's export artifacts (`kidney_efficientnetb0_final.keras`, `kidney_class_indices.json`) are structured to fit this same pattern as a fourth scan type, though — as noted in Sections 2 and 18 — its actual integration into the live deployed application is not confirmed by the materials reviewed for this revision, and the notebook's own save-cell comment referring to "your Streamlit app" does not match the project's actual Gradio-based deployment stack.

17. Libraries and Dependencies

Training (shared across notebooks): `tensorflow/keras` — model building and training; `scikit-learn` — metrics; `numpy`, `pandas` — data handling; `matplotlib`, `seaborn` — visualization; `Pillow (PIL)` — image loading; `tqdm` — progress bars; `kagglehub` — dataset download (COVID and Pneumonia notebooks); `google.colab (drive.mount)` — persistence to Google Drive.

New in the Kidney notebook: `split-folders (splitfolders)` — stratified, per-class train/validation/test directory splitting (70/15/15); `OpenCV (cv2)` — CLAHE contrast enhancement and Grad-CAM heatmap colorization/overlay; `matplotlib.cm` — colormap utilities, referenced carefully in the clinical-summary function, which computes a local confusion matrix under a distinctly named variable (`local_cm`) specifically to avoid a naming collision with the imported `matplotlib.cm` module.

Note: the kidney notebook also installs the kaggle pip package at the start of Block 1, but this package is never actually used — the dataset is loaded from a manually uploaded ZIP file already placed in Google Drive, not via the Kaggle API or kagglehub, unlike the COVID-19 and Pneumonia notebooks.

Deployment (Hugging Face Space, per the original three-model requirements.txt): `gradio (SDK 6.19.0)`, `groq`, `numpy`, `tensorflow-cpu`, `pillow`. Adding the kidney model to this deployment would additionally require `opencv-python-headless` (for the CLAHE/Grad-CAM utilities) if the explainability feature is carried into the deployed app.

18. Project Folder Structure

The notebooks and the Hugging Face Space were not provided as a single connected repository; the structure below is reconstructed from the file-saving code in the notebooks and the file list of the deployed Space, extended with the artifacts produced by the kidney notebook.

```
radiology-assistant/ | └─ notebooks/ | └─
EfficientNetB0__Brain_Tumor_MRI_Classification.ipynb | └─ covid_19.ipynb | └─
Xray_Pneumonia_EfficientNetB3.ipynb | └─ kidney_model_training.ipynb # new |
```

```

├─ results/                                     # Brain Tumor notebook output only | ─
sample_images.png | ── training_history.png | ── confusion_matrix.png | ─
class_metrics.png | ── roc_curves.png | ── models/
# Locally-saved checkpoints | ── best_brain_tumor_model.keras | ─
enhanced_sensitivity_model.keras | ── best_model.keras                                     # COVID-
19 | ── best_efficientnetb3.keras                                                     # Pneumonia | ─
best_kidney_model.keras                                                                # new – ModelCheckpoint best (Phase-2 epoch 6) |
├─ kidney_efficientnetb0_final.keras                                                    # new – final saved/downloaded model | ─
kidney_class_indices.json                                                              # new – {'Cyst':0,'Normal':1,'Stone':2,'Tumor':3}
├─ deployment/                                                                        # Hugging Face Space:
"radiology_assistant" | ── app.py                                                       # Gradio
application (MediScan AI) | ── requirements.txt | ── README.md
# Space metadata (sdk: gradio) | ── best_brain_tumor_model.keras | ── x-ray.keras
├─ Covid_19.keras

```

Datasets are not stored in the repository. The Brain Tumor and Kidney datasets are loaded via direct ZIP upload/extraction (Google Drive, in the Kidney notebook's case); the COVID and Pneumonia datasets are downloaded at runtime via `kagglehub.dataset_download()`.

A deployment-ready renamed copy of the kidney model (analogous to `x-ray.keras` / `Covid_19.keras`) and an `app.py` updated to expose a fourth scan-type option were not present in the materials reviewed for this revision; the kidney model's deployment integration is treated as a follow-on step (see Section 20).

19. Challenges Encountered

- Class imbalance in the COVID, Pneumonia, and Kidney datasets was addressed with `compute_class_weight(class_weight="balanced", ...)`; the Brain Tumor dataset avoided this issue by being pre-balanced. The Kidney dataset shows the most pronounced imbalance of the four (Stone $\approx 3.7\times$ rarer than Normal).
- Catastrophic forgetting risk during fine-tuning was mitigated in all four notebooks by using a low fine-tuning learning rate and unfreezing only part of the backbone at once — 30% for Kidney, versus roughly 58% (and later $\sim 79\%$ in an enhanced pass) for Brain Tumor.
- Specific false-negative pattern in the Brain Tumor model (pituitary tumors occasionally misclassified as "no tumor") was diagnosed via manual error analysis and addressed with an additional targeted fine-tuning pass.
- Inconsistent `val_auc` logging was observed during Pneumonia model training, while the same metric computed correctly during the final `model.evaluate()` call — a likely metric-state reset artifact.
- Class-index ambiguity for binary datasets, addressed in the deployed app with a `CT_FLIP` safety flag.
- Raw logits vs. probabilities bug in production, fixed with defensive `_as_probability(s)` helper functions.
- (New) Migrating the Grad-CAM implementation to Keras 3's stricter functional-model API required explicitly reconstructing a sub-model around the EfficientNet backbone's own input/output tensors — attempting to query an inner layer's output directly from the outer nested model would otherwise raise a `KeyError`.
- (New) A naming collision between a locally computed confusion-matrix variable and the imported `matplotlib.cm` module was avoided in the clinical-summary function by explicitly namespacing the local variable (`local_cm`).
- (New) Minor documentation/wording inconsistency: the kidney notebook's model-saving cell refers to the target deployment as "your Streamlit app," while the project's actual and previously documented deployment framework is Gradio. This appears to be a wording slip rather than an indication that the deployment stack has changed, but is flagged here for correction.
- (New) The kaggle pip package is installed in the kidney notebook's setup cell but is never used; the dataset is instead loaded from a manually uploaded Google Drive ZIP file, unlike the COVID-19 and Pneumonia notebooks' use of `kagglehub`.

20. Future Improvements

- Extend the Grad-CAM + CLAHE explainability pipeline already implemented for the Kidney model to the Brain Tumor, COVID-19, and Pneumonia models, so all four diagnoses carry a visual explanation.
- Integrate the trained Kidney model into the deployed Gradio application as a fourth selectable scan type, including updating the chatbot's supported-topics scope to cover kidney disease.

- Run the notebook's alternative EfficientNetB3 configuration (already supported via the MODEL_NAME flag) to directly compare against the EfficientNetB0 results reported here, and select whichever backbone validates better.
- Investigate the Stone-class precision gap (90.5%) — the model's single weakest metric — with further error analysis, given 20 of the 221 scans predicted as Stone were actually Normal kidneys.
- Where feasible, benchmark against the architectural techniques used in the higher-scoring published studies on this dataset (custom lightweight backbones, dedicated class-rebalancing via data trimming) to help close the gap to the state-of-the-art figures cited in Section 14.5.
- Replace the heuristic "relevant scan vs. irrelevant image" check in the Brain Tumor notebook with a dedicated, properly trained classifier.
- Resolve the CT_FLIP class-order ambiguity permanently by explicitly encoding/verifying class-to-index mapping at training time.
- Expand each binary classifier (COVID, Pneumonia) toward multi-class differential diagnosis.
- Perform k-fold cross-validation and evaluate on external, out-of-distribution datasets across all four models.
- Add persistent (database-backed) storage for patient scan history in the deployed app.
- Support DICOM file input directly, in addition to standard JPG/PNG uploads.
- Quantize/optimize the models (e.g., TensorFlow Lite) for faster CPU inference or edge/mobile deployment.
- Add automated regression tests around the app's probability-normalization logic.

21. Conclusion

This project demonstrates a complete, end-to-end applied deep learning workflow across four distinct medical imaging tasks — brain tumor MRI classification, COVID-19 CT screening, pneumonia X-ray detection, and kidney disease CT classification — using a consistent two-phase transfer-learning methodology built on EfficientNet backbones (B0, B4, B3, and B0 respectively). All four models achieved strong test-set performance (89–98% accuracy, 0.95–0.999 ROC-AUC), with particular attention paid to recall on the medically riskier error type in the Brain Tumor notebook, and to a comprehensive per-class error analysis in the Kidney notebook.

The kidney model additionally introduces visual explainability to the project via Grad-CAM, combined with CLAHE contrast enhancement for clinical readability — a capability the original three-model documentation had listed only as a future improvement, and which is now implemented for one of the four models, with extension to the remaining three identified as the natural next step.

Beyond the individual models, the project's most notable achievement remains unifying multiple independently trained classifiers into a single, polished, patient-facing web application (MediScan AI) — complete with a confidence-aware UI, a context-aware medical chatbot, and clear educational disclaimers. The documented challenges (class-order ambiguity, logits-vs-probability bugs, false-negative patterns, and — newly — Keras 3 functional-model API constraints on Grad-CAM) and their fixes reflect a realistic, iterative engineering process, and the updated future-improvements list outlines a credible path toward a more complete, clinically robust, four-model system.

Appendix: Data Sources Referenced in the Notebooks

- Brain Tumor MRI Dataset — Kaggle (brain-tumor-mri-dataset)
- SARS-CoV-2 CT-Scan Dataset — Kaggle, plameneduardo/sarscov2-ctscan-dataset
- Chest X-Ray Images (Pneumonia) — Kaggle, paultimothymooney/chest-xray-pneumonia, author Paul Mooney
- CT KIDNEY DATASET: Normal-Cyst-Tumor-and-Stone — Kaggle, nazmul0087/ct-kidney-dataset-normal-cyst-tumor-and-stone (new)

Published benchmarks referenced for context in Section 14.5 (external work, not produced by this project):

- Islam et al., "Explainable deep learning for early diagnosis of chronic kidney disease from CT images in Bangladeshi patients," Scientific Reports (2026) — the "Kid-Net" EfficientNetB3-based model, 99.30% cross-validation accuracy.

- "Fine-tuned deep transfer learning: an effective strategy for the accurate chronic kidney disease classification," PMC (2025) — fine-tuned EfficientNetV2B0, 99.73–99.75% accuracy.
- "KidneyNeXt: A Lightweight Convolutional Neural Network for Multi-Class Renal Tumor Classification in Computed Tomography Imaging," PMC (2025) — 99.96% accuracy on the Kaggle CT Kidney dataset.
- "KidneyNet: A Novel CNN-Based Technique for the Automated Diagnosis of Chronic Kidney Diseases from CT Scans," MDPI Electronics (2024) — reports EfficientNet-B1 (98.07%) and EfficientNet-B2 (98.61%) baselines on this dataset.

Not specified in the notebooks: exact license terms of each dataset, hardware specifications beyond "GPU detected," total wall-clock training time per model (per-epoch step timings are available in the execution logs, but no aggregate total is reported), and confirmation of whether the kidney model has been integrated into the live Hugging Face deployment.